

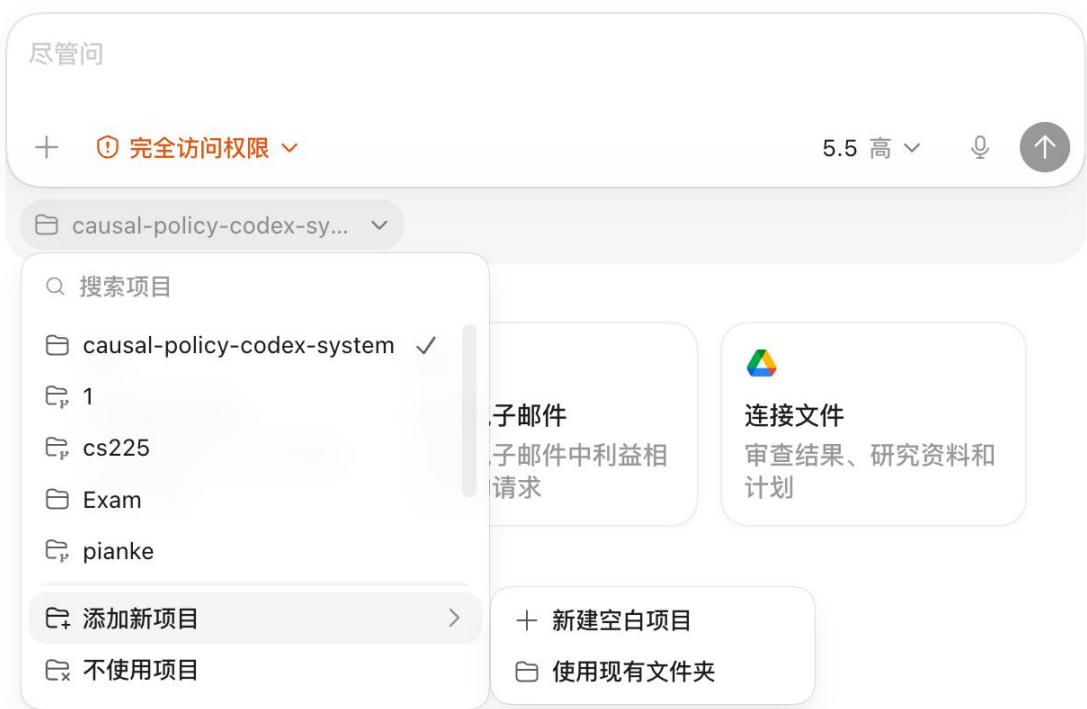
因果推断与政策评估智能体系统

基于 AI Agent 的研究设计自动化工作台

哲学院 段泽嘉 2022200779

摘要

Causal Policy Codex System: 面向因果推断与政策评估的 AI Agent 工作台，直接接收接近真实研究起点的问题，例如“某城市限行政策是否降低了 PM2.5?”“补贴政策是否影响新能源车销量?”然后把模糊的政策问题一步步压实为可检查、可复现的研究方案，并结合因果推断和前沿 causal ML 方法，拆成可执行的 skills、subagents、routing rules 和 quality gates。利用 Agent 给大语言模型搭建一个框架，使其做政策分析专家的角色扮演，按照一套真实且规范的研究流程工作。



维度	项目对应内容
做什么东西	搭建一个 AI Agent 驱动的因果推断与政策评估系统。 用户提出政策或事件问题后，系统 输出研究设计、方法推荐、政策证据链、数据方案、估计脚本、诊断与报告。
用什么工具	Codex 负责政策长文本初筛、仓库规则读取、 subagents 调度、脚本运行、估计器与管线代码生成； Python/R 负责 DID、事件研究、DML、GRF 等分析。
融入哪些方法	DID / event study、synthetic control / SDID、IV、RDD、matching、DML、AIPW、causal forest / GRF、policy learning。
达到什么效果	把一句政策问题推进到 design card、source register、analysis panel、baseline estimates、robustness checklist 和 report。

目录

1 设计目标	3
1.1 要解决的问题	3
1.2 核心思路：让 AI Agent 按研究方法论工作	3
1.3 用到的 AI 工具：Codex	3
2 系统总体架构	4
3 AI Agent 编排与协作	6
4 技能库与方法覆盖	6
5 方法路由引擎	7
6 标准工作流	7
7 政策文件处理与证据链	9
8 端到端案例：城市限行政策对 PM2.5 的影响	9
8.1 从问题到报告的操作链路	9
8.2 原始趋势：先看平行趋势	9
8.3 DID：把反事实讲清楚	10
8.4 事件研究：动态效应与预趋势诊断	11
8.5 基线回归：TWFE DID	12
8.6 稳健性、安慰剂与诊断结果	12
8.7 结果解释：能说什么，不能说什么	12
8.8 复现清单	12
9 质量把控与边界	13
10 创新点与局限	13
10.1 创新点	13
10.2 局限	13

1 设计目标

1.1 要解决的问题

政策评估看起来是一个很日常的问题：政策出台前后，指标有没有变化？但单纯的前后对比通常不够，因为政策实施的同时，宏观趋势、地区差异、选择性实施、预期效应和其他政策都可能一起变化。所以，一个合格的评估至少要回答五件事：

第一，我们想估计的到底是 ATE、ATT、CATE、dynamic effect 还是 LATE；

第二，treatment 和 outcome 怎么定义；

第三，哪些假设让反事实可以被识别；

第四，政策事实和数据口径是否可靠；

第五，诊断、稳健性和结论强度如何呈现。

这个流程很适合让 AI 工具参与，但前提是不能让 AI 自由发挥。我的设计目标就是：把 AI 放进因果推断的方法论框架中，让它按研究步骤工作，而不是看到相关性就直接写“政策有效”。

1.2 核心思路：让 AI Agent 按研究方法论工作

核心思路可以理解为：把因果研究写成 AI 可读、可执行、可复核的流程。包括四类组成部分：项目规则 AGENTS.md、可复用 skills、专职 subagents，以及 hooks 质量把控。

这样做以后，用户不需要每次都写很长的 prompt。比如只说“评估限行政策是否降低 PM2.5”，系统就会先澄清 estimand，再看是否存在处理组和对照组、政策前后、政策事实证据、数据结构和诊断条件。也就是说，AI 的第一反应被训练成先做识别，而不是先跑回归。

1.3 用到的 AI 工具：Codex

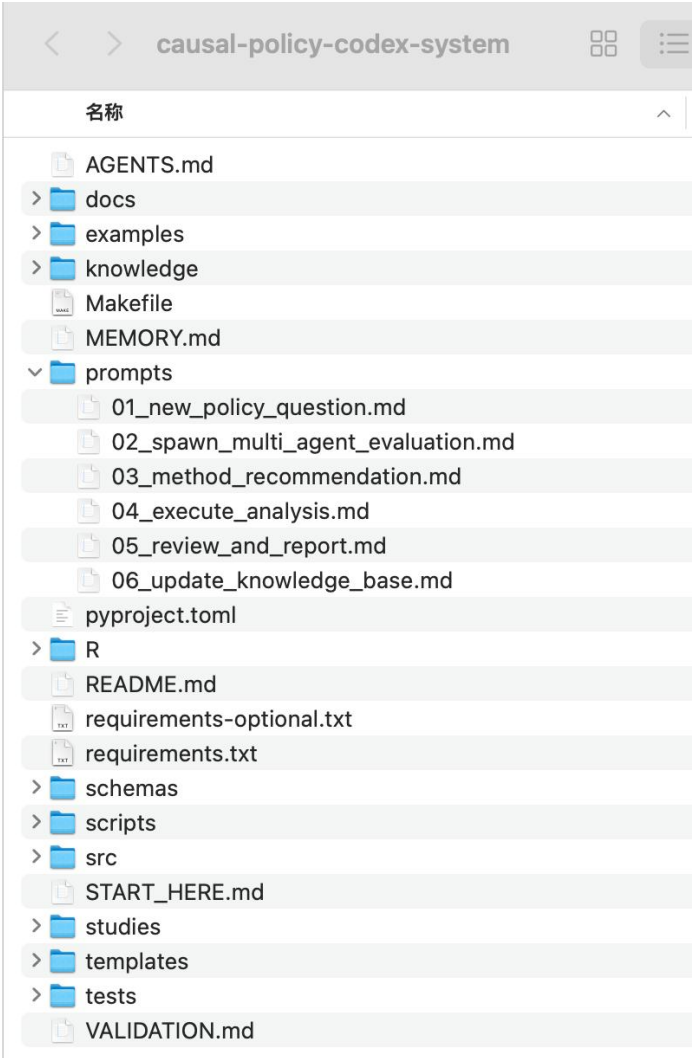
系统只使用 Codex 作为主驱动 AI 工具。Codex 负责政策长文本和资料初筛，读取仓库规则，发现 skills，调用 scripts 和 CLI，调度 subagents，生成估计器与数据管线代码，并把 method router 的第一反应纳入可复核的研究流程。Python/R 仍作为可执行分析环境，负责 DID、事件研究、DML、GRF 等模型运行与结果输出。

工具	在系统中的位置	主要作用
Codex	主驱动智能体与研究工作台	读取政策长文本和 AGENTS.md，发现 skills，调用 subagents，运行 CLI / scripts，生成估计器、数据管线、design card 和报告。
Python / R	可执行估计层	运行 DID、事件研究、DML、GRF 等分析脚本，输出表格和图。
规则化 method router	方法推荐入口	根据问题文本和数据结构给出主方法、备选方法、关键假设和诊断

2 系统总体架构

系统搭建采用七层架构：最上面是抽象的研究原则，越往下越接近可以执行的代码、数据和报告。这样的结构有一个好处：AI Agent 不会只看到一个孤立脚本，而是能看到研究问题、方法规则、政策证据和最终交付物之间的关系。

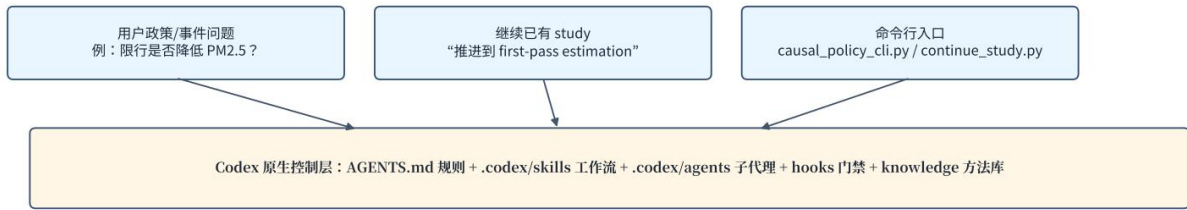
层级	组件	作用
1	项目规则 AGENTS.md	定义角色定位、12 步因果路线图、方法路由规则、质量把控和报告标准。
2	.codex/skills/	15 个可复用 workflow，可显式调用，也可由智能体自动匹配。
3	.codex/agents/	10 个专职 subagents，分别处理政策来源、文件抽取、识别、方法、数据、估计、稳健性和政策转化。
4	.codex/hooks/	拦截危险命令、自动注入研究上下文、进行命令复盘。
5	knowledge/	存放方法矩阵、前沿方法、政策来源清单、事件库和政策文件处理协议。
6	scripts/ + src/	方法路由器、估计器、政策抽取、design card 校验、顶层调度器和 CLI。
7	studies/	每个具体研究的工作区，包括 design card、数据、图表、报告和复现包。



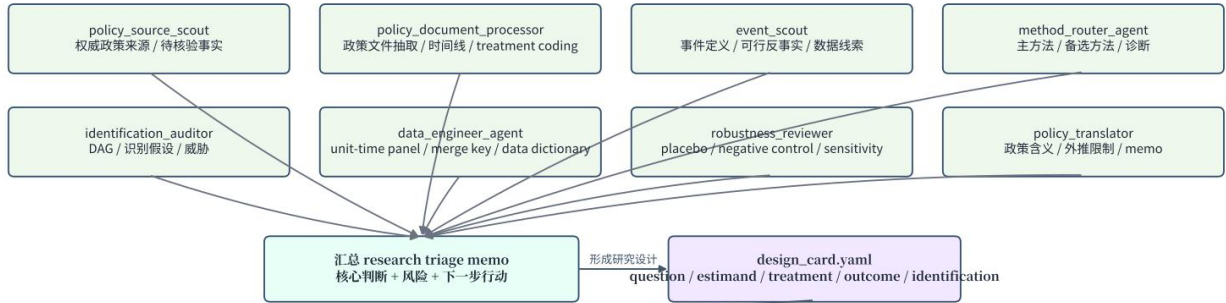
Causal Policy Codex Agent：从政策问题到研究产物的整体流程

基于仓库中的 AGENTS.md、.codex/skills、.codex/agents 与 study_orchestrator.py 整理

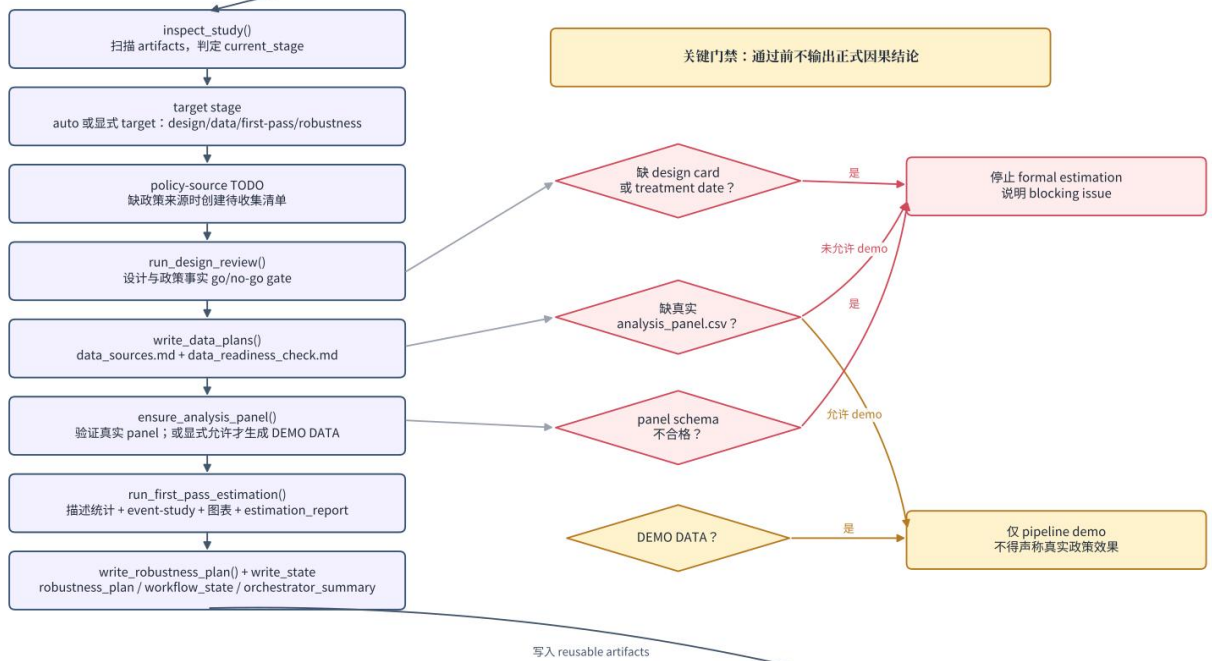
① 入口与项目控制层



② 新问题：多 Agent 并行研究分工



③ 继续已有 study：Study Orchestrator 的确定性推进



④ 主要输出产物



图 1. 从政策问题到研究产物的整体流程：入口、Agent 分工、调度器、质量把控与最终 artifacts。

3 AI Agent 编排与协作

系统把 AI 拆成不同角色。一次完整政策评估会由 `study_orchestrator` 作为中枢，然后并行调度多个 `subagent`。每个 `subagent` 只负责一类判断，最后再汇总。这样做比单一模型直接回答更稳，因为不同环节的错误更容易被发现。

Subagent	解释
<code>policy_source_scout</code>	先找权威政策来源，避免从二手新闻直接编码 <code>treatment</code> 。
<code>policy_document_processor</code>	读取本地政策文件，抽取发布时间、生效时间、实施范围、豁免和处理强度。
<code>event_scout</code>	寻找事件定义、时间线、候选数据源和类似研究背景。
<code>identification_auditor</code>	站在因果识别角度审查 DAG、混杂、选择性、溢出和预期效应。
<code>method_router_agent</code>	把问题结构映射到 DID、SCM、IV、RDD、DML 等方法。
<code>data_engineer_agent</code>	设计 <code>unit-time panel</code> 、 <code>merge key</code> 、数据字典和可复现数据管线。
<code>estimator_agent</code>	在 <code>design card</code> 和数据列确定后实现 Python / R 估计脚本。
<code>robustness_reviewer</code>	扮演“反对者”，提出 <code>placebo</code> 、敏感性、替代窗口、聚类标准误等检验。
<code>policy_translator</code>	把技术结果翻译成带注意事项的政策解释，避免过度声称。
<code>study_orchestrator</code>	识别已有 <code>study</code> 的阶段，并把它推进到下一步。

4 技能库与方法覆盖

Skill 可以理解为把高频研究任务封装成一个可调用流程。比如政策文件处理、DID 事件研究、DML、因果森林和复现报告，都被做成了独立 skill。

研究阶段	对应 Skills
政策事实	<code>policy-source-scout</code> ; <code>policy-document-intake</code> ; <code>policy-document-processor</code> ; <code>event-research-scout</code>
设计与识别	<code>policy-evaluation-intake</code> ; <code>causal-dag-identification</code> ; <code>causal-method-router</code>
估计方法	<code>did-event-study</code> ; <code>synthetic-control</code> ; <code>iv-rdd-design</code> ; <code>dml-doubly-robust</code> ; <code>causal-forest-policy-learning</code>
审查与交付	<code>robustness-sensitivity</code> ; <code>replication-report</code>
流程编排	<code>advance-study-stage</code> : 一句话推进已有研究

方法上，系统覆盖了从经典准实验到 `causal ML` 的 9 类设计。

方法	适用场景	核心假设 / 诊断
RCT / 实验	处理可以随机分配或随机鼓励。	随机分配、SUTVA、依从性；可估计 ITT / TOT。
DID / 事件研究	处理前后 + 处理/对照 + 面板。	平行趋势、无预期、无溢出；检查 <code>pre-trends</code> 。
合成控制 / SDID	单一或少数处理单位，政策前序列较长。	<code>donor pool</code> 能拟合处理前轨迹；检查权重和拟合。
工具变量 IV	处理内生，但有外生变异来源。	相关性、排除限制、单调性；关注 LATE 和 <code>weak IV</code> 。
断点回归 RDD	阈值或分数线决定处理资格。	<code>cutoff</code> 附近不可操纵、潜在结果连续；带宽敏感性。
匹配 / 倾向得分	选择主要由可观测变量解释。	条件可忽略、重叠、平衡诊断。

Double ML / AIPW	高维可观测混杂， 需要灵活 nuisance model。	正交矩、cross-fitting、overlap；减少机器学习偏误。
因果森林 / GRF	关注异质性 CATE。	honesty、重叠、独立验证；避免过度外推。
Policy Learning	关注资源配置、targeting 或政策规则。	welfare 定义透明、约束明确、独立评估集。

5 方法路由引擎

方法路由器是一个透明的规则系统。它先对问题文本做关键词打分，比如“面板”“政策前后”“对照组”会把问题推向 DID；“阈值”“分数线”会推向 RDD；“随机试点”会推向实验或鼓励设计。然后它再结合结构化元数据，例如是否有处理组、对照组、时间维度、政策前序列长度、高维协变量等。只给第一反应，不直接给最终结论。method_router_agent 和 identification_auditor 会继续检查制度背景、数据可得性和识别威胁。换句话说，系统把“推荐方法”和“证明因果识别成立”分开处理。

路由线索	优先推荐	为什么
随机分配 / 随机试点	RCT / encouragement design	最接近实验设计，可以直接利用随机变异。
阈值 / 分数线规则	RDD	cutoff 附近处理状态发生跳变，可比较阈值两侧单位。
外生工具 / 鼓励设计	IV / 2SLS	用工具变量隔离处理中的外生部分。
单一处理单位 + 长政策前序列	Synthetic Control / SDID	用 donor pool 构造反事实轨迹。
面板 + 政策前后 + 对照	DID / event study	用对照组趋势近似处理组反事实变化。
高维可观测混杂	DML / AIPW / Matching	机器学习处理复杂控制变量，但仍需重叠和可忽略性。
异质性 / targeting	Causal Forest / Policy Learning	估计 CATE 并进一步形成政策分配规则。
无可信对照或数据不足	数据采集 + 识别改进计划	不强行跑模型，先说明缺口。

6 标准 workflow

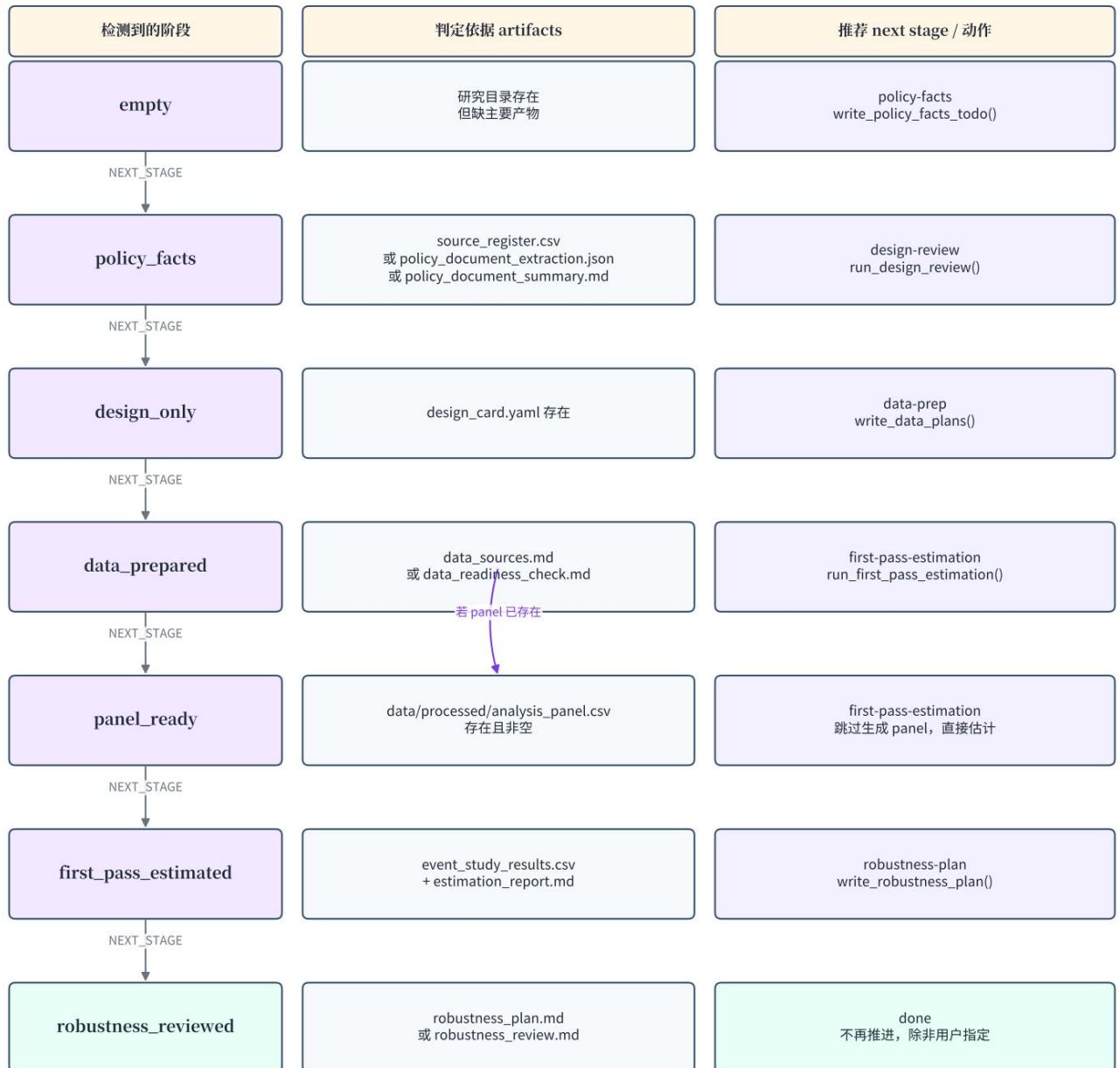
无论问题多复杂，都先按 12 步路线图推进。这样可以防止 AI 直接从一句政策问题跳到回归结果。

步骤	要检查的内容
1 Question	政策/事件、目标群体、时间、地点、分析单位
2 Estimand	ATE、ATT、CATE、dynamic effect、LATE、policy value
3 Treatment & Outcome	处理定义、结果变量、测量窗口
4 DAG / Mechanism	混杂、机制、选择、溢出、预期效应
5 Identification	让反事实可识别的假设
6 Policy Sources	权威文件、时间线、证据日志
7 Data Plan	单位、时间、协变量、缺失、版本记录
8 Method	主方法、备选方法、为什么不是其他方法
9 Diagnostics	pre-trends、overlap、first stage、placebo 等
10 Robustness	替代窗口、替代结果、聚类、敏感性
11 Interpretation	因果/描述区分、外部有效性、异质性
12 Reproducibility	脚本、seed、数据字典、报告与复现说明

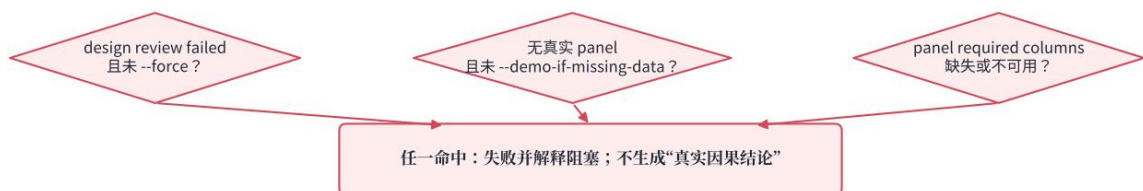
对于已经建好的研究，系统还提供 7 阶段状态。用户只需要说“继续这个 study，推进到 first-pass estimation”，study_orchestrator 就会检查当前 artifacts，判断是否缺少 design card、policy facts 或 analysis panel，然后决定能否推进。

Study Orchestrator：阶段识别与推进状态机

由 src/causal_policy_system/study_orchestrator.py 的 STAGE_ORDER / NEXT_STAGE / continue_study() 抽象而来



估计前停止条件



每次推进后都会写入：study_state.yaml、workflow_state.json、workflow_state.md、logs/orchestrator_run_*.json、orchestrator_summary.md / orchestrator_report.md

图 2. Study Orchestrator 阶段识别与推进状态。

7 政策文件处理与证据链

政策评估最容易出错的地方，其实往往不是模型，而是政策事实。比如政策到底是发布日、正式生效日，还是实际执行日开始算？是否有试点、豁免、分阶段执行？如果这些事实没有核验，后面的 treatment 编码就会出错。

因此系统在估计之前强制做 policy source register 和 policy document extraction。自动抽取只作为第一遍，关键日期、适用范围和执行强度必须保留不确定项，并在人工复核后写入 design card。

输入	处理动作	输出
政策原文、实施细则、地方通知	Codex 调用 policy_document_processor 初步抽取政策事实，并整理 source register。	source_register.csv; policy_document_extraction.json; policy_document_summary.md
不同来源的政策日期	区分发布日、有效日、实施日、修订日；保留冲突。	policy timeline; treatment timing 备选方案
政策范围与强度	抽取地区、对象、豁免、执行机制和强度指标。	treatment definition intensity coding 建议
未核验或冲突信息	标记为 blocking 或 non-blocking issue。	design_review.md; 识别风险说明

8 端到端案例：城市限行政策对 PM2.5 的影响

为了展示完整链路，我使用了一个合成 demo：24 个城市、36 个月、6 个处理城市，政策在第 20 个月实施，真实效应设为 -2.0。特别说明：只是为了验证工作流和图表产出，不构成任何真实政策结论。

8.1 从问题到报告的操作链路

1. 明确政策问题：某城市交通限行政策是否降低 PM2.5？
2. 定义 estimand：处理城市在政策实施后的平均处理效应，近似为 post-period ATT。
3. 构造 city-month 面板：结果变量为 pm25，处理变量为 treatment = treated_ever × post。
4. 方法路由：因为有处理组、对照组、政策前后和面板结构，主方法选择 DID，并用 event study 检查动态效应和预趋势。
5. 先画 raw trends，再计算四格 DID，把反事实终点讲清楚。
6. 运行 TWFE DID，并加入城市固定效应、月份固定效应和天气控制。
7. 补充 event study、placebo timing、窗口敏感性和协变量敏感性。
8. 输出可复现 artifacts：数据、JSON、CSV 中间表、图表和报告。

8.2 原始趋势：先看平行趋势

DID 的第一步不是直接跑回归，而是先画 raw trends。图中可以看到处理组和对照组在政策前大致同向变化，政策后处理组均值更低。但这只是目视检查，还不能单独证明平行趋势成立。

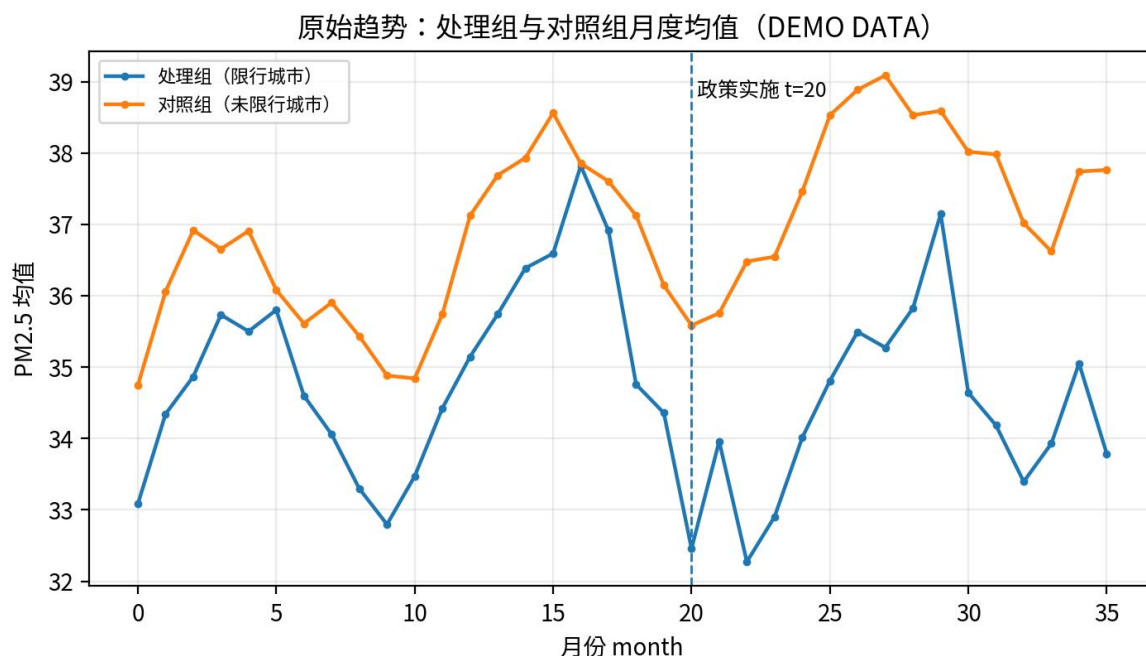


图 3. 原始趋势图 (DEMO DATA)：处理组与对照组 PM2.5 月度均值。

8.3 DID：把反事实讲清楚

DID 的直觉是：如果没有限行政策，处理组在政策后的变化应该和对照组类似。于是我们把对照组的前后变化平移到处理组，得到处理组的反事实终点。实际终点和反事实终点之间的差，就是 DID 估计。

序列	时期	PM2.5 均值	角色	说明
对照组	政策前	36.490	观测值	未限行城市政策前均值
对照组	政策后	37.536	观测值	用于估计共同时间变化
处理组	政策前	34.984	观测值	限行城市政策前均值
处理组	政策后	34.321	观测值	实际观察到的政策后均值
处理组反事实	政策后	36.031	反事实	处理组政策前均值 + 对照组前后变化
DID	政策后	-1.709	效应	实际处理组政策后-反事实处理组政策后

计算过程：对照组变化 = $37.536 - 36.490 = 1.047$ ；处理组反事实政策后 = $34.984 + 1.047 = 36.031$ ；DID = $34.321 - 36.031 = -1.709$ 。

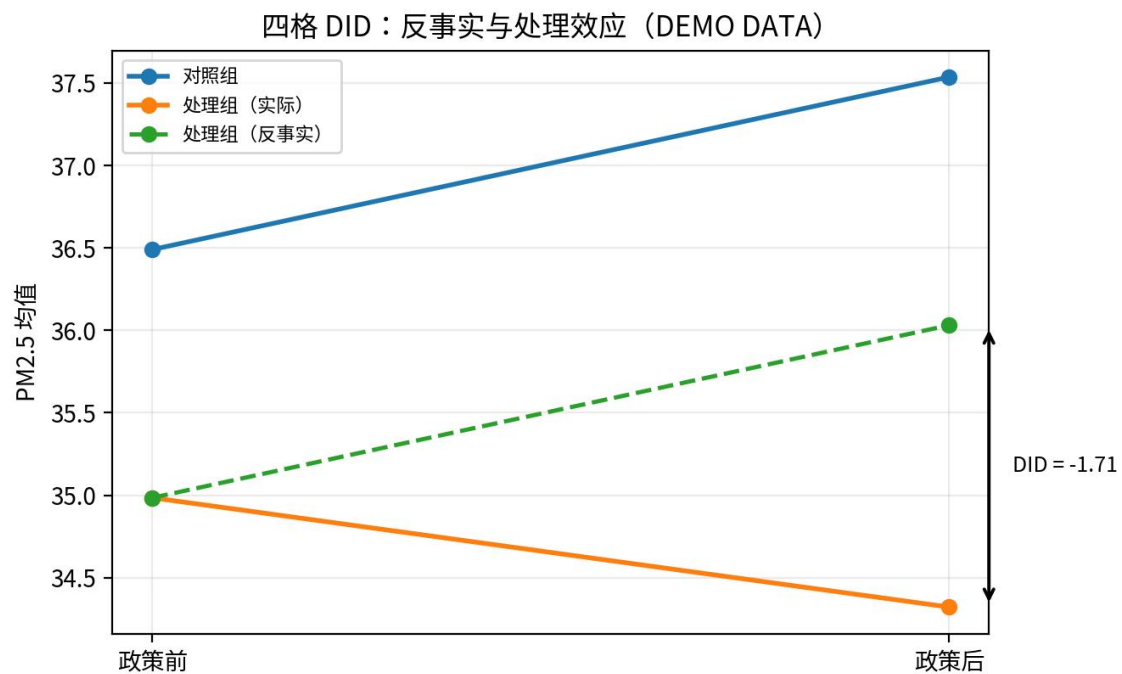


图 4. 四格 DID (DEMO DATA)：对照组变化被平移为处理组反事实，本 demo 中 DID 为 -1.71。

8.4 事件研究：动态效应与预趋势诊断

事件研究的作用是把 DID 拆成相对政策实施时间的动态系数。它不仅能展示政策后效应如何变化，也能检查政策前系数是否接近 0。这个 demo 中政策前系数大多围绕 0 波动，但置信区间较宽，说明诊断必须和点估计一起呈现。

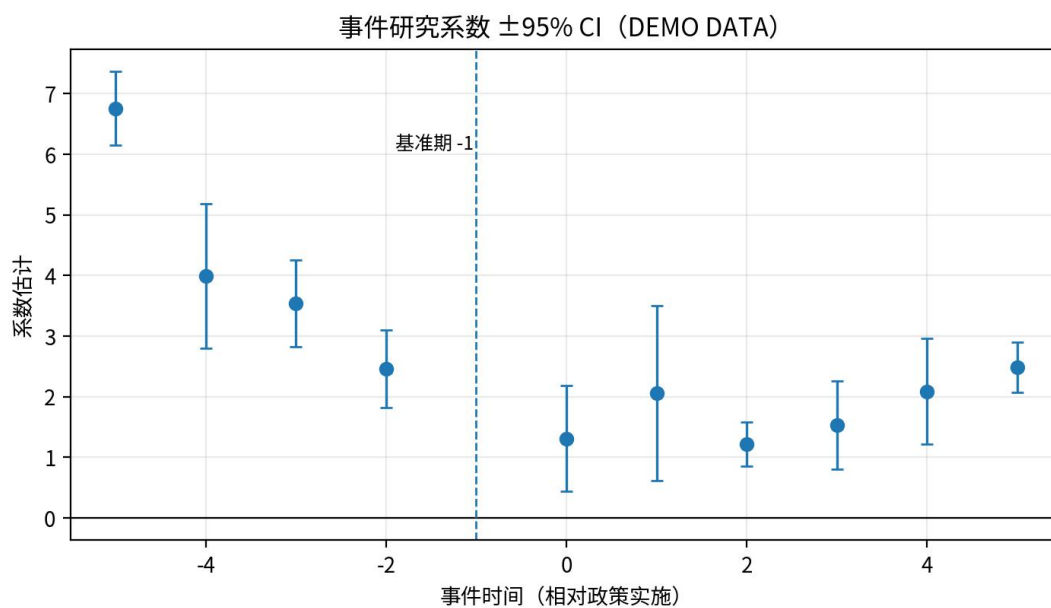


图 5. 事件研究系数 ±95% CI (DEMO DATA)：政策前系数提示预趋势风险。

8.5 基线回归：TWFE DID

在四格 DID 之后，系统进一步运行两维固定效应回归： $pm25 \sim treatment + C(city) + C(month) + weather_index$ 。这个设定控制了城市不随时间变化的差异、所有城市共同面对的月份冲击，以及 demo 中模拟的天气指标。TWFE 估计值为 -1.765，聚类标准误为 0.195，样本量为 864。

8.6 稳健性、安慰剂与诊断结果

完整案例不能只放一个回归表，还要展示检查链路。下面把主要估计、协变量敏感性、窗口敏感性和 placebo timing 放在一起。

检查	估计值	标准误	95% CI	解释
四格 DID（全样本）	-1.709			主展示估计；没有模型标准误，仅作为透明计算。
TWFE + 天气控制	-1.765	0.195	[-2.147, -1.384]	加入城市固定效应、月份固定效应和天气控制。
TWFE 不含天气控制	-1.709	0.232	[-2.164, -1.255]	检查结果是否主要由天气协变量设定驱动。
仅保留政策后 8 个月	-1.982	0.260	[-2.492, -1.473]	检查短期窗口是否仍为负向。
提前 6 个月 placebo	0.365	0.327	[-0.277, 1.006]	只使用真实政策前数据；理想情况下应接近 0。

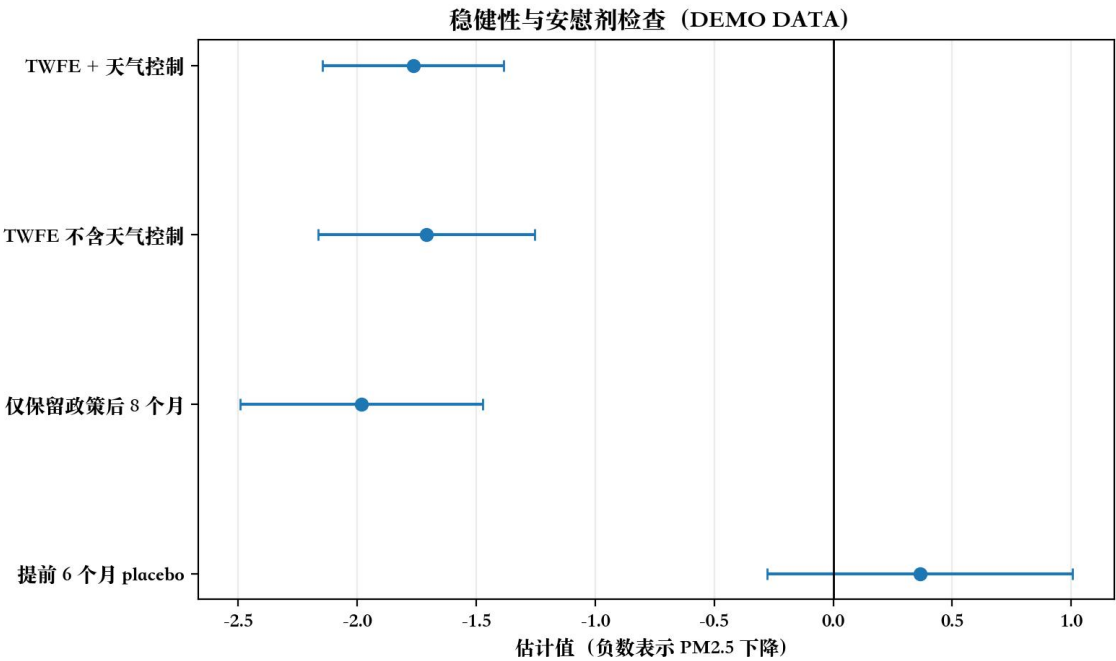


图 6.稳健性与安慰剂检查 (DEMO DATA)：负值表示 PM2.5 下降，placebo 用于诊断。

8.7 结果解释：能说什么，不能说什么

在这个 demo 中，四格 DID 为 -1.709，TWFE DID 为 -1.765，都指向政策后处理组 PM2.5 相对下降。因为数据生成时真实效应为 -2.0，结果用于说明 workflow 能够恢复大致方向。

真实研究还需要核验政策发布日期、生效日、执行强度、豁免规则、监测站口径、气象冲击、产业变化、跨城市污染转移和相邻城市溢出。如果预趋势诊断不过关，应该降级为设计讨论或重新寻找对照组，而不是强行解释 DID。

8.8 复现清单

- 运行命令：python scripts/run_demo_analysis.py。

- 输入数据: `examples/data/demo_panel.csv`。
- 估计结果: `examples/outputs/demo_estimates.json`。
- 中间表: `demo_did_cells.csv`、`demo_event_study.csv`、`demo_robustness.csv`。
- 图表: `demo_raw_trends.png`、`demo_four_cell_did.png`、`demo_event_study.png`、`demo_robustness.png`。
- 报告: `examples/outputs/demo_report.md`。

9 质量把控与边界

AI 可以加速研究设计、资料整理和代码实现，但不能自动创造可信的反事实。系统内置了 hooks 和报告标准。

- 危险命令拦截: `pre_tool_use_policy.py` 阻止删除 `studies/`、`raw data` 等关键文件。
- 上下文注入: `user_prompt_context.py` 在政策评估类提示中自动加入因果路线图和调度器提示。
- 结论强度标注: 报告需要标注 `strong / moderate / weak / design-only`。
- 绝不杜撰: 政策日期、法条名称、样本量、数据来源和引用必须核验。
- demo 与真实分离: 合成数据只能用于 workflow 演示，不能作为真实政策证据。

10 创新点与局限

10.1 创新点

- 把因果推断路线图、方法选择和报告标准编码为 AI Agent 可执行的 `skills / agents / hooks`。
- 透明 method router 给第一反应，再由识别审计和制度背景修正，兼顾速度与可信度。
- `study_orchestrator` 能自动识别已有研究阶段，并推进到 `policy facts`、`design review`、`data prep`、`first-pass` 或 `robustness`。
- `claim strength`、demo/真实分离、证据链和 `blocking issue` 机制，从源头限制过度结论。
- 把 DID、event study、SCM、IV、RDD、DML、GRF、policy learning 变成同一系统内的可选方法族。

10.2 局限

第一，demo 数据只能证明管线能跑通，不证明真实政策有效。

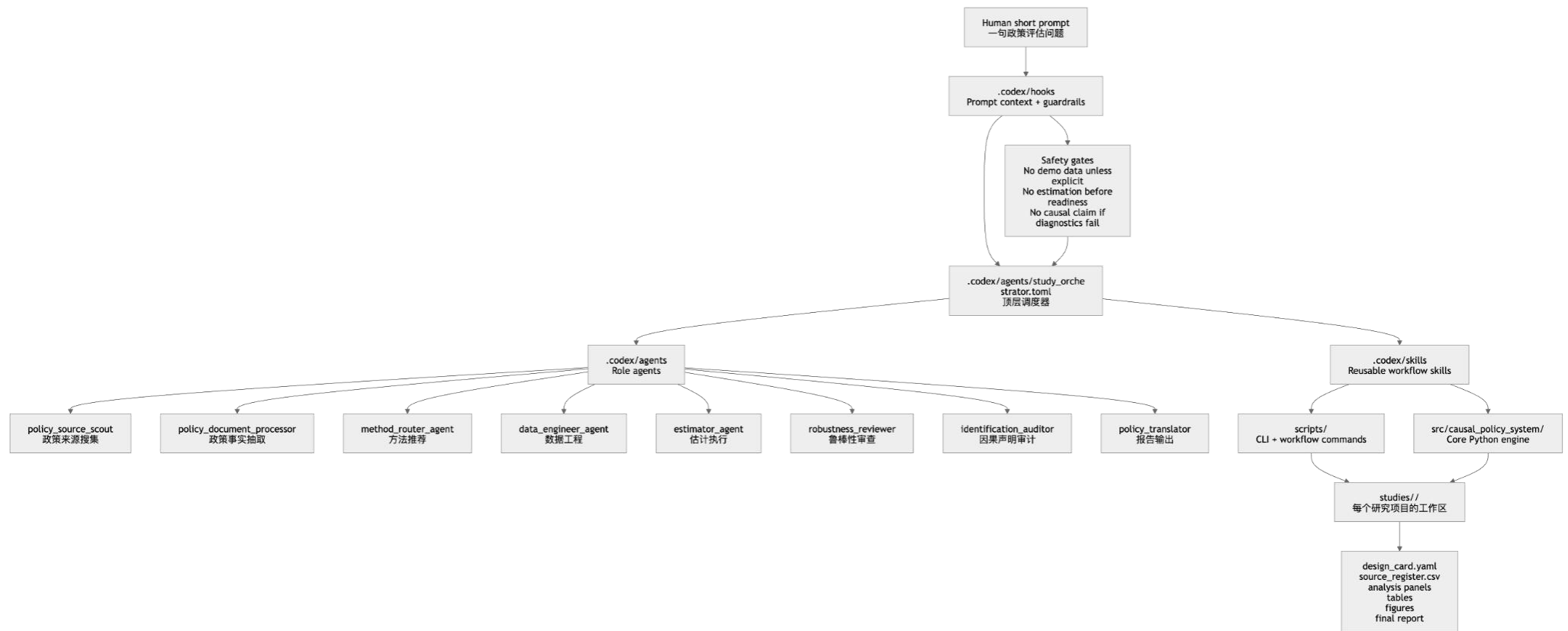
第二，真实政策评估需要接入官方数据，并核验制度细节、政策执行和数据口径。

第三，若存在 `staggered adoption`，传统 TWFE 可能有权重问题，需要使用 Callaway-Sant' Anna、Sun-Abraham 或其他稳健估计。

第四，空间溢出、一般均衡效应和政策执行异质性仍需要专门设计，不能简单交给通用 DID。

下一步改进可以集中在三方面：接入真实政策数据源；把 `staggered DID`、`SDID`、`matrix completion` 和 `DML/GRF` 的诊断做得更完整；建立自动化但可审计的 `evidence log` 和 `reproducibility package`。

这张图来自系统初步流程图，展示了短 prompt、hooks、safe gates、study orchestrator、role agents、skills、scripts、src 和 studies 工作区之间的关系。



系统组件拓扑图：从 human short prompt 到 agents、skills 和 studies artifacts。